

Tmux Is All You Need

A Terminal-Multiplexer Architecture for Recursive Coding-Agent Orchestration

Fabio Pauli¹

apritzclass@gmail.com

Abstract

The dominant paradigm for supervising autonomous coding agents is *recurrent*: a single human operator context-switches between one agent at a time, carrying task state forward in their own working memory, dispatching instructions in sequence. This design is inherently sequential, which precludes parallelization across agents and makes the human the bottleneck and the single point of failure. We propose the *Tmux Orchestration Architecture*, a control substrate based solely on the terminal multiplexer `tmux`, dispensing with recurrence in the human entirely. In our architecture an *orchestrator* agent attends directly to any worker agent in the swarm through a small set of content- and address-based primitives — `send-keys`, `capture-pane`, `pipe-pane`, hooks, and `wait-for` — replacing the human relay on every coordination hop with addressed reads and writes on stable pane IDs. We are precise about what this buys: transport and supervision become cheap, durable, and scriptable, while the semantic work of orchestration remains exactly where it was, in the orchestrator. Because a `tmux` server persists independently of any attached client, and because a pane can itself run a `tmux` client, the architecture is *recursive*: an orchestrator can spawn sub-orchestrators, forming a tree of coordination while keeping the human attached at the root, in the loop, *eventually*. We compare orchestration substrates on separated metrics — human interventions in the inner loop, semantic re-emissions per data hop, transport to a known agent, and discovery cost — and describe the primitives, the topology, and the control loop in enough detail to reproduce a semi-autonomous, human-supervised agent swarm.

1 Introduction

Recurrent supervision of coding agents — Claude Code, Codex, and similar LLM-driven development tools — has firmly established itself as the state of the art in single-operator workflows. The operator opens one agent, states a task, waits, reads the result, forms the next instruction, and repeats. State is carried forward as a hidden representation h_t inside the operator’s own head,

$$h_t = f(h_{t-1}, x_t),$$

where x_t is the t -th agent interaction and h_t is the operator’s evolving mental model of the project. This recurrent formulation, in its various guises, has remained the boundary of what a single human can manage. It has two fundamental limitations, both familiar from the recurrent sequence-modeling literature [3, 1]:

1. **Sequential computation.** The operator can advance only one interaction per step. Two agents cannot be truly supervised at once; attention is time-sliced, not parallel. Throughput is bounded by human cycle time, not by the number of available agents or cores.
2. **The relay.** To route a result produced by agent A into the working context of agent B , the operator must read A , hold the result in working memory, context-switch, and re-type it to B . The hop count is constant — $A \rightarrow \text{relay} \rightarrow B$ — but every hop crosses the single-threaded hidden state h_t : it is serialized behind all other hops, paid at human latency, and lossy in exactly the way h_t is lossy. Long dependency chains degrade through repeated re-encoding much as long-range dependencies degrade an RNN.

Attention mechanisms [1] dissolved the analogous problem in sequence modeling by allowing any position to be reached from any other in a constant number of operations, and the Transformer [4] showed that once you have attention, *recurrence is unnecessary*. We make the equivalent claim for agent orchestration. We propose the *Tmux Orchestration Architecture*, a model architecture eschewing human recurrence and relying entirely on a terminal multiplexer to draw global dependencies between agents. `Tmux` allows significantly more parallelization; a single orchestrator can drive a swarm of coding agents, routing outputs to inputs directly, with the human elevated from the inner loop to a supervisory gate.

The contributions of this paper are: a mapping from the components of self-attention onto concrete `tmux` primitives (§3), including scaled selection (§3.2.1), multi-head orchestration (§3.2.2), and positional encoding via stable pane

identity and event time (§3.5); a “Why Tmux” analysis (§4) comparing orchestration substrates on separated metrics — human interventions, semantic re-emissions, transport, and discovery — rather than a single overloaded “path length”; and a description of how to deploy, batch, schedule, and regularize a live swarm (§5), including the human-in-the-loop gate as an explicit regularizer.

2 Background

The goal of reducing sequential human involvement forms the basis of numerous tools: shell job control, `make -j`, CI runners, and multi-agent frameworks that coordinate LLM calls through a central Python process. In most of these, the number of operations required to relate signals from two distant workers still grows with the distance between them, or coordination is funneled through a single orchestrating process that must itself parse, buffer, and re-emit every message — re-introducing a recurrent bottleneck one level up.

Self-attention, an attention mechanism relating different positions of a single sequence, has been used successfully to replace recurrence outright. To the best of our knowledge the Tmux Orchestration Architecture is the first coordination substrate to rely entirely on a terminal multiplexer — a program whose original purpose was to let *one human* keep *many* shells alive across disconnects — to instead let *one agent* keep *many agents* alive, addressable, and mutually routable, with the human as an optional attached client rather than a mandatory relay. Two properties of tmux make this possible and distinguish it from a plain subprocess pool:

- **Client/server separation.** A tmux *server* owns the sessions; *clients* merely attach to view and drive them. Detaching a client does not kill the work. The swarm’s state is durable with respect to the human’s presence — the human may detach, sleep, and reattach, and the computation persists. This is the substrate analog of a residual connection carrying state around each interaction (§3.1).
- **Programmatic control.** Beyond the interactive UI, tmux exposes a fully scriptable command surface and a machine-readable *control mode* (`-C/-CC`) in which every event is reported as a structured, %-prefixed message on a stream [7]. An agent, not just a human, can therefore be a first-class tmux client.

3 Model Architecture

Most competitive orchestration schemes have an encoder–decoder structure. Here, the *encoder* is the set of **worker agents** that map a task specification to a representation (a working tree, a diff, a set of captured outputs). The *decoder* is the **orchestrator agent** that, given those representations, generates the next instructions one at a time, autoregressively consuming its own prior decisions. The Tmux Orchestration Architecture follows this overall structure using stacked panes, content- and address-based attention, and per-agent feed-forward reasoning, shown schematically in Figure 1.²

3.1 Session, Window, and Pane Stacks

The encoder and decoder are composed of a stack of identical structural layers, provided by tmux’s three-level hierarchy:

- **Server** — one per socket (`-L name / -S /path/socket`), a namespace holding all sessions. Distinct sockets give separate servers whose sessions, channels, and options cannot collide — *namespace separation*, which must not be mistaken for a sandbox. Every server runs as the same user on the same filesystem with the same credentials, and any process of that user may connect to any socket it can name. Confining an untrusted agent takes an OS boundary (a separate user, container, or VM) and credential scoping; tmux contributes tidiness, not security.
- **Session** — a durable collection of windows, detachable from any client. One session per project or major objective.
- **Window** — a full-screen layer within a session, tiled into panes. One window per orchestration *head* (§3.2.2) or per phase.
- **Pane** — a single pseudo-terminal running exactly one agent process. The pane is the atomic unit: the residence of one Claude Code or Codex session.

²We assume tmux ≥ 3.2 throughout: pane-scoped user options, `list-panes` filters, and `display-popup` are comparatively recent arrivals. On older servers the architecture degrades gracefully to fewer heads and more `grep`.

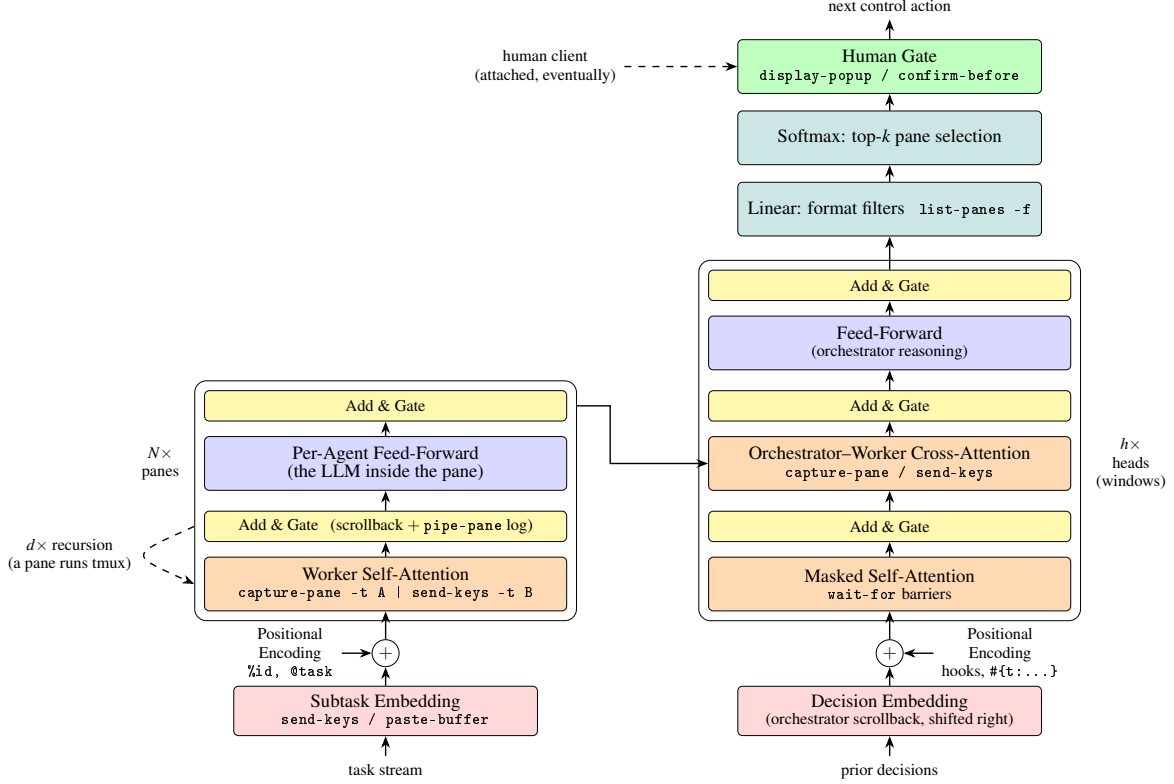


Figure 1: The Tmux Orchestration Architecture, drawn in the family style. Worker panes (left, the encoder stack) and the orchestrator (right, the decoder) exchange information through `capture-pane/send-keys` cross-attention; `wait-for` masks the orchestrator’s self-attention; the human gate sits where the softmax used to be.

Each worker pane is wrapped by two connections analogous to the residual connection and layer normalization around each Transformer sub-layer. A *residual path*: the pane’s scrollback and, optionally, an append-only log via `pipe-pane -o -t <pane> 'cat > log'`, which streams output to a file for the pane’s lifetime, so context flows *around* each interaction: $\text{LayerOutput} = \text{Agent}(x) + x$, where x is the durable prior context. And a *normalization gate*: a checkpoint at which the orchestrator or human inspects and, if necessary, constrains the pane’s output before it propagates — the human-in-the-loop gate of §5.4, bounding an agent’s divergence before it affects shared state. The output of each sub-layer is thus $\text{Gate}(x + \text{Agent}(x))$, with *Agent* implemented by the LLM inside the pane and *Gate* by the review policy.

3.2 Attention

An orchestration attention function maps a *query* (the orchestrator’s current subgoal) and a set of *key-value* pairs (the addressable worker panes and their captured contents) to an output (the next control action). Let the swarm be a set of panes $P = \{p_1, \dots, p_n\}$. Each pane exposes, via *tmux format variables*, a **key** k_i (its addressable, queryable metadata) and a **value** v_i (its current content):

```
k_i = ( #{pane_id}, #{@role}, #{@task_id}, #{@state},
        #{pane_current_command}, #{pane_title},
        #{pane_dead}, #{window_silence_flag} )
v_i = capture-pane -p -t "#{pane_id}"
```

The address in every key is `pane_id` — the immutable %N the server assigns at creation — never a window/pane *index*, which is a presentation coordinate that renumbers under topology changes (§3.5). The keys are cheaply enumerable for the whole swarm in a single call:

```
tmux list-panes -a -F "#{pane_id} role=#{@role} state=#{@state} |
```

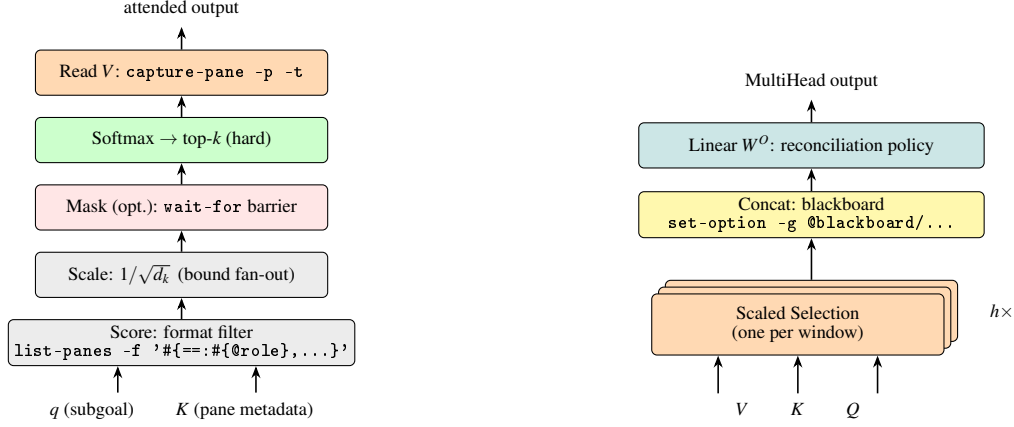


Figure 2: (left) Scaled Selection. (right) Multi-Head Orchestration: h selection channels run in parallel windows and are combined through a shared blackboard.

```
cmd=#{pane_current_command} dead=#{pane_dead}'
```

3.2.1 Scaled Dot-Product Attention (Scaled Selection)

We call our particular attention “Scaled Selection.” The input consists of a query q (the current subgoal), keys k_i of dimension d_k (pane metadata), and values v_i (pane contents). We compute a relevance score between the query and each pane’s key, apply a softmax to obtain weights, and read the weighted values (Figure 2, left).³

$$\text{OrchAttention}(q, K, V) = \sum_i \text{softmax}_i \left(\frac{\text{score}(q, k_i)}{\sqrt{d_k}} \right) \cdot \text{Capture}(p_i). \quad (1)$$

In practice $\text{score}(q, k_i)$ is realized as a *format filter* — a predicate over the pane’s key fields that the orchestrator evaluates with `list-panes -f / if-shell -F`. For example, “attend to reviewer workers currently blocked at a shell prompt”:

```
tmux list-panes -a \
-f '#{@@:#{role},reviewer},#{state},idle}' |
-F '#{pane_id}'
```

The scaling factor $1/\sqrt{d_k}$ is where the analogy is at its most decorative, and we say so plainly: multiplying every score by the same positive constant changes no ranking, so it cannot change what any hard top- k selects. What the factor guards in the original — selection staying sharp as dimensionality grows — is real here, but it is enforced by choosing k , the per-step fan-out, small and fixed. In deployment the “softmax” is a Boolean format predicate over pane keys followed by a capped number of reads: filter, then bound. Where genuine ranking is wanted (most-recently-quiet first, longest-blocked first), the orchestrator computes a numeric score per pane and sorts — `tmux` supplies the fields (`#{t:...}` timestamps, user options), not the ordering. Reading a value is `capture-pane`; writing to the selected panes is `send-keys`, with a paste-buffer path for large or special-character payloads — one uniquely named buffer per task, deleted on paste, so concurrent dispatches cannot race on a shared buffer name:

```
tmux capture-pane -p -t "$target" -S -200          # READ (the "V")
tmux send-keys -t "$target" "run the tests" Enter  # WRITE (route subgoal)
tmux load-buffer -b "task-$id" ./subtask-prompt.txt # large payload, no escaping
tmux paste-buffer -d -b "task-$id" -t "$target"     # unique buffer, freed on paste
```

Two practical notes. First, `send-keys` interprets key names — `Enter`, `C-c` — so literal payloads should be sent with `-l`, lest a string containing “Enter” be helpfully pressed on the worker’s behalf. Second, keys land in whatever the pane is doing *right now*; dispatch is therefore gated on the worker being at rest (§3.5), since injecting a subtask into an agent mid-generation adds noise without gradient.

³The vector space in which terminal scrollbar admits convex combination is left as future work; in deployment the softmax temperature is taken to zero and the sum collapses to hard selection.

3.2.2 Multi-Head Orchestration

Rather than run a single orchestration channel over the full swarm, we found it beneficial to run h channels in parallel — one per tmux **window** — each with its own selection projection (its own `@role` filter, subset of the swarm, and subgoal). Each head i attends independently (Figure 2, right),

$$\text{head}_i = \text{OrchAttention}\left(qW_i^Q, KW_i^K, VW_i^V\right), \quad (2)$$

where the projections W_i are, concretely, the per-window scoping of which panes and key fields that head considers relevant. Multi-head orchestration lets the architecture jointly attend to different parts of the project at different phases: head 1 supervises implementation panes, head 2 a test/CI pane, head 3 documentation, head 4 a long-running build. We are careful about where the concurrency lives: the *workers* under each head run concurrently as independent OS processes, but a single orchestrator walking the windows remains one decision process, time-slicing its own attention — a window is an organizational scope, not a thread. Decision-level parallelism requires one orchestrator process per head, which is exactly the recursion of §4: a head that matters enough is given its own sub-orchestrator pane. The heads are combined by concatenation into a shared **blackboard** — persistent key/value state in tmux user-options, readable and writable by any head:

```
tmux set-option -g @blackboard/build_status "green" # write (Concat)
tmux show-option -gv @blackboard/build_status      # read
```

MultiHead = Concat(head₁, ..., head_h) W^O , where W^O is the reconciliation policy that resolves conflicting writes (last-writer-wins, or an orchestrator-mediated merge).

3.2.3 Applications of Attention in Our Model

The architecture uses orchestration attention in three ways. **(1) Orchestrator→worker cross-attention:** the decoder queries over all worker panes, reading their captured state and writing their next instruction — the analog of encoder-decoder attention, via addressed reads and writes on stable pane IDs. **(2) Worker self-attention:** workers attend to one another *without* routing through the orchestrator when a direct pipe is authorized, `capture-pane -p -t A | ... | send-keys -t B`. **(3) Masked orchestrator self-attention:** the orchestrator attends to its own prior decisions (scrollback, blackboard) to remain autoregressive, but is *masked* from depending on subtasks that have not yet completed. The mask is enforced with `wait-for`: a barrier that blocks the orchestrator from consuming a result until the producing worker signals completion (§3.3), preserving the auto-regressive property.

3.3 Position-wise Feed-Forward Networks (Per-Agent Reasoning and Barriers)

In addition to attention sub-layers, each pane contains a fully-connected feed-forward network applied to that position independently: the LLM inference performed *inside* the agent. This is the actual “thinking” — identical in form at every pane but with different, per-pane parameters (the agent’s own context window and system prompt). Synchronization between the attention layer and these per-agent computations is provided by `wait-for`, tmux’s built-in barrier/signal channel:

```
tmux wait-for -S subtask-42 # worker: signal completion (last line of its task)
tmux wait-for subtask-42   # orchestrator/dependent: block until signalled
```

`wait-for -L / -U` additionally provide a lock/unlock primitive for mutual exclusion over the shared working tree, preventing two workers from writing the same files concurrently.

Two properties deserve emphasis. First, a signal on a channel with no waiter is not lost: the server latches one wake, and the next waiter consumes it. But it is a *one-bit latch*, not a counting queue — early signals coalesce, and a reused channel can hand a stale wake to the wrong consumer. The discipline that follows is one globally unique channel per task attempt (`done:$swarm:$task:$attempt`), never reused. Second, channels are server-global, so recursive sub-swarms take their own sockets (§3.1) and collisions become impossible by construction. `wait-for -L/-U`, finally, is an *advisory* mutex among cooperating scripts: nothing compels a worker to take it, and a crashed holder releases nothing, so a real lock carries an owner record and recovery logic beside it.

3.4 Embeddings and Readout

Before a subtask enters the swarm it is *embedded* into the substrate: a natural language objective is tokenized into a shell-injectable instruction and a target address (which pane, which role). Symmetrically, the *readout* projects a pane’s raw terminal output back into a decision-usable representation via `capture-pane` (with `-e` to retain escape sequences or `-J` to rejoin wrapped lines). As in the Transformer, we share the same learned mapping between the input embedding (how we phrase instructions to agents) and the pre-readout transformation (how we parse their replies): the orchestrator’s prompt conventions and its output parser are two directions of one shared protocol, which for control-mode clients is the %-prefixed structured stream itself [7].

A caveat keeps this section honest. `capture-pane` returns the rendered grid and retained history — a picture of a terminal, not a message stream — and `pipe-pane` admits one pipe per pane, streaming only while attached. For an operator’s console this is exactly right; as the sole data plane of an autonomous protocol it lacks framing, acknowledgement, and schema. Deployment therefore moves payloads over a thin per-worker file protocol with atomic renames (§5.1) and reserves the terminal for what it is: an attention surface, not a message bus.

3.5 Positional Encoding

The swarm, as described, is a *set* of panes — orchestration attention is permutation-invariant and by itself carries no notion of which agent came first or which subtask depends on which. Since dependency order matters, we must inject information about the position of each agent in the topology and in time.

Tmux supplies identity and order natively, provided one uses the right coordinates. Each pane has a **unique ID** (`{pane_id}`, the %N the server assigns at creation), immutable for the pane’s lifetime; sessions and windows carry %N and @N likewise. Window and pane *indexes*, by contrast, are presentation coordinates: they renumber under `break-pane`, `join-pane`, `swap-pane`, and `move-window` — the very operations §5 uses to reshape the swarm mid-run. Scripts therefore address panes by ID and keep semantic position in explicit per-pane user options (`@role`, `@task_id`, `@state`); indexes are for the human’s fingers. We combine identity with a **temporal coordinate** from event hooks and the `{t:...}` time formats, so each agent carries a position signal analogous to the sinusoidal encoding, letting the orchestrator reason about dependency order without a recurrent scan:

$$\text{PE}(\text{pane}) = (\text{pane_id}, \text{@task_id}, \text{timestamp of last state change}).$$

The temporal component is maintained *event-drivenly* by hooks rather than by polling:

```
# Stamp when a pane goes quiet (a watchdog event, not a completion signal)
tmux set-hook -g alert-silence \
  'set-option -p @last_silent "${t:window_activity}" ; run-shell "notify-orch ${pane_id}"'
# React to a worker crashing (a "position" removed from the sequence)
tmux set-hook -g pane-died \
  'run-shell "orch handle-death ${pane_id} status=${pane_dead_status}"'
```

Available hook events include `pane-died`, `alert-activity`, `alert-silence`, `alert-bell`, `session-created`, `client-attached`, and `client-detached` [8]. Note that `pane-died` fires only when `remain-on-exit` is on, leaving the corpse addressable for a `capture-pane` post-mortem before `respawn-pane`; with the default off the pane closes and the weaker `pane-exited` fires — the evidence leaves with it. Activity/silence monitoring is enabled with `monitor-activity on` and `monitor-silence <seconds>` — these are *window* options, and `alert-silence` fires per window, so a per-worker silence watchdog wants one window per worker, which the multi-head layout already provides. And it is a *watchdog*, never a completion protocol: silence cannot distinguish a finished worker from one mid-inference, blocked on the network, deadlocked, or waiting for confirmation — and `pane_current_command` is only a corroborating hint, since a TUI agent stays in the foreground whether generating or idle. Completion is therefore explicit: the worker’s last act is to set `@state done` and signal its unique per-attempt channel (§3.3); silence merely tells the orchestrator where to look when nothing has been said for too long. This gives the swarm a *positional* sense of “who just spoke and who just went quiet,” computed once and event-drivenly, rather than re-derived by a sequential human sweep.

4 Why Tmux

In this section we compare the tmux orchestration substrate to alternatives — recurrent human management and centralized-process orchestration. An earlier draft compressed the comparison into a single “path length”; an adversarial reviewer correctly observed that $A \rightarrow \text{relay} \rightarrow B$ is a constant number of hops however slow the relay, and that a

Table 1: Separated coordination metrics. “Semantic re-emissions” counts the times a payload crosses a deciding process (a mind or a model) to move between two agents; transport assumes the target’s pane ID is already known; discovery is a single `list-panes` round trip that enumerates all n panes server-side.

Substrate	Human ops, inner loop	Sem. re-emissions / hop	Transport (known agent)	Discovery
Recurrent human management	every hop	1 (the operator)	serial re-type	$O(n)$ visual scan
Central-process orchestrator	none	1 (the process)	$O(1)$ IPC	process-dependent
Tmux orchestration (this work)	gate only	0 on authorized pipes	$O(1)$ addressed, <code>-t %id</code>	$O(n)$ scan, 1 round trip

substrate scan such as `list-panes -f` is linear in the panes inspected however few round trips it takes. We therefore separate the metrics: *human interventions* required in the inner loop; *semantic re-emissions* per data hop, the number of times a payload must cross a deciding process — a mind or a model — to move between two agents; *transport cost* to reach an agent whose address is already known; and *discovery cost* to find out which agent that is.

The honest reading of Table 1 is that tmux buys the two middle columns. A data hop between workers whose IDs are known (`capture-pane -t %4 | ... | send-keys -t %7`) crosses no deciding process at all, where recurrent management pays the operator’s working memory on every hop and a central-process design pays a parse–decide–re-emit cycle. What tmux does *not* buy is any reduction in semantic work: it selects no recipients, interprets no dependencies, reconciles no conflicts, and trusts no output — every decision still happens in the orchestrator, whose reasoning costs are identical across substrates and appear in no column. Tmux replaces terminal-switching and process-management labor, not judgment.

An adversarial reader will object that the tmux server is itself a single central process. It is — and that is the point of the middle column. The server moves bytes between pseudo-terminals without parsing them, holds no conversation state, and re-emits nothing through a context window, so a hop through it counts zero semantic re-emissions. The charge against the central-process design is not that it is central but that it is *semantically* central: every payload crosses its parse–decide–re-emit cycle whether or not that payload needed a decision.

Two further benefits motivated the choice of tmux. **Durability/detachability:** because the server outlives its clients, the human can detach entirely and the swarm keeps running; work is not lost on disconnect. This is what lets the human be in the loop *eventually* (§5.4) rather than *always*. **Recursion:** a pane may run a tmux client that owns its own server or session, so an orchestrator can spawn *sub-orchestrators*, each managing a sub-swarm; the addressing composes via nested prefixes (`C-b C-b ...`) and per-socket namespaces. Depth- d recursion yields a coordination *tree*; the human attaches only at the root and can literally `choose-tree` to watch the whole hierarchy.

5 Deployment

This section describes how a swarm is decomposed, batched, scheduled, and regularized in operation — the “training regime” of the architecture.

5.1 Task Decomposition and Batching

Each engineering objective is decomposed by the orchestrator into subtasks batched by *independence*: subtasks with no shared file dependency are dispatched to distinct worker panes in the same step and proceed in parallel; dependent subtasks are serialized behind `wait-for` barriers (§3.3) and `wait-for -L` locks over shared paths. Batches are sized to keep fan-out k bounded (§3.2.1) so neither the orchestrator’s control bandwidth nor the human’s review bandwidth saturates.

Two further disciplines carry the batching in practice. Each worker owns a separate Git *worktree* — a shared index turns independent subtasks into merge conflicts you scheduled for yourself — and payloads travel through a thin per-worker *shim*: task and result envelopes as files delivered by atomic rename, with the pane’s terminal carrying the agent’s face rather than the freight. Tmux wakes, supervises, and displays; the files remember.

5.2 Hardware and Schedule

A swarm runs on a single host (or a `tmux -S` socket shared over SSH), one pane per worker process. The orchestrator loop alternates between an *event-driven* phase — blocking on hooks and `wait-for` signals, consuming near-zero

CPU while workers think — and a *dispatch* phase triggered when a pane goes silent, dies, or signals completion. Control mode (-cc) is used when the orchestrator is itself an agent: it reads the %-prefixed notification stream (%output, %window-pane-changed, %exit, ...) directly rather than screen-scraping [7]. A production control-mode client is real engineering, not a line-oriented loop: it must correlate %begin/%end/%error responses with the commands that caused them, tolerate escaped and non-UTF-8 output, honor the protocol’s flow control so a slow reader does not fall behind, and resynchronize with `capture-pane` after a disconnect. We use the notification stream for wakes and supervision, keep payloads on the file protocol of §5.1, and declare a fully general control-mode parser future work.

5.3 Optimizer (the Control Loop)

The orchestration policy is the “optimizer” driving the swarm toward the objective. We use an event-driven loop with bounded polling as a fallback:

```
loop:
  wait on { hooks(pane-died, alert-silence), wait-for signals }    # event-driven
  on signal from pane p:
    v <- capture-pane(p)                                           # attend (read)
    decision <- orchestrator.step(q, v)                           # per-agent FFW of the decoder
    apply(decision):                                              # attend (write)
      send-keys / paste-buffer to selected panes                 # continue work
      respawn-pane p                                             # restart a crashed worker
      break-pane / join-pane / swap-pane                         # reshape topology
      set-option @blackboard/...                                 # update shared state
    if decision.needs_human: escalate_to_gate()                   # see 5.4
```

We adopt a schedule analogous to warmup-then-decay: early in an objective the orchestrator polls more aggressively and escalates to the human more readily (*warmup* — establishing shared context and trust); as the objective stabilizes it relies more on event-driven hooks and widens the human-gate interval (*decay*). `respawn-pane/respawn-window` provide restart-on-failure, and `break-pane/join-pane` let the optimizer restructure the swarm topology mid-run.

5.4 Regularization (Human-in-the-Loop Gate)

We employ several regularizers to prevent the swarm from overfitting to a locally plausible but globally wrong plan. **Human-gate:** the single most important regularizer, and the easiest to fake. A popup is an interface, not an enforcement mechanism — with no client attached there is nowhere to draw it, and a worker holding push credentials needs nobody’s permission. The gate is therefore a *capability boundary*. Workers run without credentials for protected operations (a push, a deploy, a destructive command); a request for one is persisted to disk and the swarm moves on; the human reviews pending requests on attach, with `display-popup` and `choose-tree` as how the queue looks, not why it holds; and a broker that alone holds the credentials validates and executes the exact approved command. Direct worker-to-worker pipes (§3.2.3) carry data, never authority. The human is thus in the loop *eventually and where it matters*, not in every inner iteration — the analog of applying regularization at the layer boundaries rather than to every activation. **Silence dropout:** panes silent beyond a threshold are treated as dropped for the current step (`monitor-silence`), forcing the orchestrator not to rely on any single worker always being available. **Checkpoint/residual:** `pipe-pane` logs and periodic git commits provide the residual path that lets the swarm recover context after a crash or a bad step.

6 Results

We report qualitative and illustrative operational results; a rigorous benchmark of swarm throughput across task suites is left to future work, and the figures below are illustrative of the regime rather than a controlled measurement. The benchmark that would settle it is well-defined — dispatch and result latency, false-completion rate under the silence watchdog, recovery time after worker and orchestrator crashes, and merge-conflict rate of shared trees versus per-worker worktrees — and this section’s claims should be read as bounded by it.

Orchestration throughput. Replacing the human relay with addressed pane operations removes the human from the inner loop, so wall-clock throughput can scale with *available parallel workers* rather than *human cycle time* — for genuinely independent tasks, and only until something else binds: review bandwidth at the gate (§5.4), CPU, API rate limits, repository contention, or the test infrastructure. Parallelism is a ceiling, not a guarantee. Within it, a single

Table 2: Variations on the Tmux Orchestration Architecture. h = orchestration heads (windows), k = per-step fan-out, d = recursion depth.

	h	k	d	Human gate	Notes
base	3	4	1	on irreversible ops	balanced; recommended default
(A) single-head	1	4	1	same	simpler; loses phase separation
(B) wide fan-out	3	12	1	same	saturates review bandwidth; thrash
(C) deep recursion	3	4	3	root only	powerful; hardest to supervise
(D) no gate	3	4	1	off	fastest and least safe; not advised

operator supervises a swarm whose aggregate rate is set by k concurrent agents rather than by one, minus everything the previous sentence lists.

Architecture variations. Table 2 varies the number of heads h , per-step fan-out k , recursion depth d , and the human gate. Removing the scaling discipline on fan-out (row B) degrades supervisory quality — the operator cannot review 12 parallel decision streams — and removing the human gate entirely (row D) maximizes raw speed while forfeiting the regularization that keeps the swarm aligned with intent. Deep recursion (row C) is the most capable configuration but concentrates oversight at the root, recommended only once trust in the lower levels is established.

Generalization to heterogeneous agents. Because the substrate addresses *panes*, not model APIs, the architecture generalizes across agent types without modification: a pane may hold Claude Code, a Codex session, a plain REPL, or a build process, all attended to through the same `send-keys/capture-pane` interface. The orchestrator need not know what runs inside a pane, only its address and its `role` — the substrate is model-agnostic.

7 Conclusion

We presented the Tmux Orchestration Architecture, a model for supervising coding-agent swarms based entirely on the terminal multiplexer, replacing recurrent human management with direct, addressable attention between agents. For orchestration tasks, a single agent driving a tmux swarm connects any two workers in $O(1)$ operations, removes the human from the inner loop, and — by virtue of the server outliving its clients and panes being able to run tmux themselves — supports detachable, recursive, semi-autonomous operation with the human retained as a supervisory gate. We plan to apply it to swarms larger than a single host (federating over `tmux -S` sockets and SSH), to learned selection policies that replace hand-written format filters, and to tighter human-gate ergonomics. We make one claim, plainly, and scope it honestly: for the *supervision layer* — keeping many coding agents alive, addressable, observable, and interruptible under one human — **tmux is all you need**. For state, transport, and authority, tmux is precisely what you should not use; knowing which layer you are standing in is the architecture.

Acknowledgements. To every worker pane that went silent at exactly the right moment, and to the human who stayed in the loop, eventually.

References

- [1] D. Bahdanau, K. Cho, Y. Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR, 2015.
- [2] S. Hochreiter, J. Schmidhuber. *Long Short-Term Memory*. Neural Computation, 1997.
- [3] I. Sutskever, O. Vinyals, Q. V. Le. *Sequence to Sequence Learning with Neural Networks*. NeurIPS, 2014.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin. *Attention Is All You Need*. NeurIPS, 2017.
- [5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, Y. Cao. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR, 2023.

- [6] N. Marriott et al. *tmux — terminal multiplexer; tmux(1) manual page*. <https://man7.org/linux/man-pages/man1/tmux.1.html>
- [7] tmux project. *Control Mode*. tmux/tmux Wiki. <https://github.com/tmux/tmux/wiki/Control-Mode>
- [8] tmux project. *Hooks and Notifications*. tmux/tmux documentation.

A Minimal Reproducible Swarm

A minimal two-worker swarm — session, per-worker windows, stable pane IDs, watchdogs, and explicit completion — expressed entirely in tmux. It is a supervisor and a console, not a full orchestrator: scheduling policy, the worker shim of §5.1, and the credential broker of §5.4 live outside it. A maintained version, with a pane registry, event log, and demo mode, ships alongside this paper as `swarm/swarm.sh`.

```
#!/usr/bin/env bash
set -euo pipefail
SOCK="swarm"; LOG="$PWD/logs"; mkdir -p "$LOG"
T() { tmux -L "$SOCK" "$@"; }

# 1. Durable, detached session; corpses stay addressable (pane-died needs it).
T new-session -d -s proj -n orch
T set-option -g remain-on-exit on
T set-option -p -t proj:orch.0 @role orchestrator

# 2. One window per worker (per-worker silence watchdog); record stable pane IDs.
for w in 0 1; do
  id=$(T new-window -dP -t proj -n "w$w" -F '#{pane_id}')
  T set-option -p -t "$id" @role worker ; T set-option -p -t "$id" @state idle
  T pipe-pane -o -t "$id" "cat >> '$LOG/worker-$w.log'"
  T set-option -w -t "proj:w$w" monitor-silence 20 # watchdog, NOT completion
  T send-keys -t "$id" -l 'claude' ; T send-keys -t "$id" Enter
  echo "$w $id" >> "$LOG/registry" # index -> immutable %id
done

# 3. Watchdog events land in an append-only log the orchestrator tails.
T set-hook -g alert-silence "run-shell \"echo silence #{pane_id} >> '$LOG/events'\""
T set-hook -g pane-died "run-shell \"echo died #{pane_id} #{pane_dead_status} >> '$LOG/events'\""

# 4. Address by %id; completion is explicit, unique per attempt, never inferred.
dispatch() { T send-keys -t "$1" -l "$2" ; T send-keys -t "$1" Enter ; }
read_pane(){ T capture-pane -p -J -t "$1" -S -200 ; }
await() { T wait-for "done:$1" ; } # one-bit latch: one channel per attempt
finish() { T set-option -p -t "$TMUX_PANE" @state done
  T wait-for -S "done:$1" ; } # a worker's last act, run in its pane

# 5. The human attaches at the root; detaching stops nothing. The gate of 5.4
# is a credential broker, not a function this script could contain.
# tmux -L swarm attach -t proj # choose-tree (C-b s / C-b w) to watch heads
```